

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

PRACTICAL - 2

Practical 1(A)

Aim :- Implement the RSA algorithm for public-key encryption and decryption, and explore its properties and security considerations. **(CLI)**

Code :-

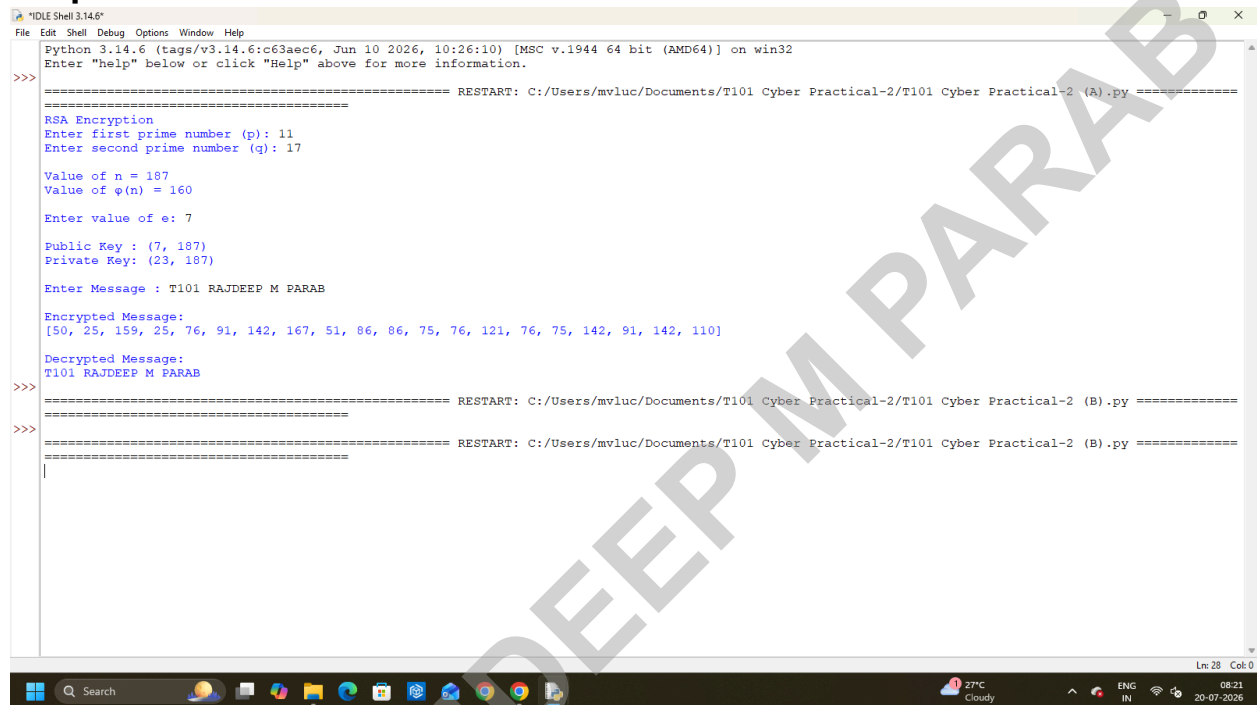
```
# RSA Encryption and Decryption (CLI)
from math import gcd
# Calculate modular inverse
def mod_inverse(e, phi):
    for d in range(2, phi):
        if (d * e) % phi == 1:
            return d
    return None
# Encrypt message
def encrypt(message, e, n):
    encrypted = []
    for ch in message:
        encrypted.append(pow(ord(ch), e, n))
    return encrypted
# Decrypt message
def decrypt(cipher, d, n):
    text = ""
    for num in cipher:
        text += chr(pow(num, d, n))
    return text
print("RSA Encryption")
p = int(input("Enter first prime number (p): "))
q = int(input("Enter second prime number (q): "))
n = p * q
phi = (p - 1) * (q - 1)
print("\nValue of n =", n)
print("Value of  $\phi(n)$  =", phi)
while True:
    e = int(input("\nEnter value of e: "))
    if gcd(e, phi) == 1:
        break
    else:
        print("e must be coprime with  $\phi(n)$ . Try again.")
d = mod_inverse(e, phi)
print("\nPublic Key :", (e, n))
print("Private Key:", (d, n))
message = input("\nEnter Message : ")
cipher = encrypt(message, e, n)
print("\nEncrypted Message:")
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

```
print(cipher)
plain = decrypt(cipher, d, n)
print("\nDecrypted Message:")
print(plain)
```

Output :-



The screenshot shows a Windows desktop with a taskbar at the bottom. The main window is 'IDLE Shell 3.14.6'. The shell displays the output of a Python script. The script performs RSA encryption and decryption. The input message is 'T101 RAJDEEP M PARAB'. The encrypted message is a list of integers: [50, 25, 159, 25, 76, 91, 142, 167, 51, 86, 86, 75, 76, 121, 76, 75, 142, 91, 142, 110]. The decrypted message is 'T101 RAJDEEP M PARAB'. The script is run from the file 'C:/Users/mvluc/Documents/T101 Cyber Practical-2/T101 Cyber Practical-2 (A).py'. The shell also shows the command prompt history and the current directory.

```
Python 3.14.6 (tags/v3.14.6:cf63a6c6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/mvluc/Documents/T101 Cyber Practical-2/T101 Cyber Practical-2 (A).py =====
RSA Encryption
Enter first prime number (p): 11
Enter second prime number (q): 17

Value of n = 187
Value of φ(n) = 160
Enter value of e: 7

Public Key : (7, 187)
Private Key: (23, 187)

Enter Message : T101 RAJDEEP M PARAB

Encrypted Message:
[50, 25, 159, 25, 76, 91, 142, 167, 51, 86, 86, 75, 76, 121, 76, 75, 142, 91, 142, 110]

Decrypted Message:
T101 RAJDEEP M PARAB

>>>
===== RESTART: C:/Users/mvluc/Documents/T101 Cyber Practical-2/T101 Cyber Practical-2 (B).py =====
>>>
===== RESTART: C:/Users/mvluc/Documents/T101 Cyber Practical-2/T101 Cyber Practical-2 (B).py =====
|
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

Practical 1(B)

Aim :- Implement the RSA algorithm for public-key encryption and decryption, and explore its properties and security considerations. **(GUI)**

Code :-

```
from tkinter import *
from tkinter import messagebox
from math import gcd
# Find modular inverse
def mod_inverse(e, phi):
    for d in range(2, phi):
        if (d * e) % phi == 1:
            return d
    return None
# Encrypt
def encrypt():
    try:
        p = int(entry_p.get())
        q = int(entry_q.get())
        e = int(entry_e.get())
        message = entry_message.get()
        n = p * q
        phi = (p - 1) * (q - 1)
        if gcd(e, phi) != 1:
            messagebox.showerror("Error", "e must be coprime with  $\phi(n)$ ")
            return
        d = mod_inverse(e, phi)
        encrypted = [pow(ord(ch), e, n) for ch in message]
        lbl_public.config(text=f"Public Key : ({e}, {n})")
        lbl_private.config(text=f"Private Key : ({d}, {n})")
        lbl_cipher.config(text="Cipher : " + str(encrypted))
    except:
        messagebox.showerror("Error", "Invalid Input")
# Decrypt
def decrypt():
    try:
        p = int(entry_p.get())
        q = int(entry_q.get())
        e = int(entry_e.get())
        n = p * q
        phi = (p - 1) * (q - 1)
        d = mod_inverse(e, phi)
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

```
cipher = entry_cipher.get()
cipher = cipher.split(",")
cipher = [int(i.strip()) for i in cipher]
plain = ""
for num in cipher:
    plain += chr(pow(num, d, n))
```

```
lbl_plain.config(text="Plain Text : " + plain)
```

except:

```
messagebox.showerror("Error", "Enter cipher correctly.")
```

```
root = Tk()
```

```
root.title("RSA Encryption & Decryption")
```

```
root.geometry("700x620")
```

```
root.configure(bg="#E8F4FA")
```

```
Label(root,
```

```
    text="RSA Encryption & Decryption",
```

```
    font=("Arial", 18, "bold"),
```

```
    bg="#E8F4FA",
```

```
    fg="navy").pack(pady=10)
```

```
Frame1 = Frame(root, bg="#E8F4FA")
```

```
Frame1.pack()
```

```
Label(Frame1, text="First Prime Number
```

```
(p)", bg="#E8F4FA").grid(row=0, column=0, padx=10, pady=8)
```

```
entry_p = Entry(Frame1, width=30)
```

```
entry_p.grid(row=0, column=1)
```

```
Label(Frame1, text="Second Prime Number
```

```
(q)", bg="#E8F4FA").grid(row=1, column=0, padx=10, pady=8)
```

```
entry_q = Entry(Frame1, width=30)
```

```
entry_q.grid(row=1, column=1)
```

```
Label(Frame1, text="Public Exponent
```

```
(e)", bg="#E8F4FA").grid(row=2, column=0, padx=10, pady=8)
```

```
entry_e = Entry(Frame1, width=30)
```

```
entry_e.grid(row=2, column=1)
```

```
Label(Frame1, text="Message", bg="#E8F4FA").grid(row=3, column=0, padx=10, pady=8)
```

```
entry_message = Entry(Frame1, width=30)
```

```
entry_message.grid(row=3, column=1)
```

```
Button(root,
```

```
    text="Encrypt",
```

```
    command=encrypt,
```

```
    bg="green",
```

```
    fg="white",
```

```
    width=18).pack(pady=10)
```

```
lbl_public = Label(root, text="", bg="#E8F4FA", font=("Arial", 11))
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 Cyber & Information Security Practical

```
lbl_public.pack()
lbl_private = Label(root,text="",bg="#E8F4FA",font=("Arial",11))
lbl_private.pack()
lbl_cipher = Label(root,text="",bg="#E8F4FA",font=("Arial",11))
lbl_cipher.pack(pady=10)
Label(root,
    text="Enter Cipher Numbers\n(separate by comma)",
    bg="#E8F4FA").pack()
entry_cipher = Entry(root,width=60)
entry_cipher.pack()
Button(root,
    text="Decrypt",
    command=decrypt,
    bg="blue",
    fg="white",
    width=18).pack(pady=10)
lbl_plain = Label(root,text="",bg="#E8F4FA",font=("Arial",12,"bold"))
lbl_plain.pack(pady=10)
root.mainloop()
```

Output :-

